

MIT Art Design and Technology University
MIT School of Computing, Pune



23CSEXXX - Natural Language Processing

Class - L.Y. (SEM-I), <Division>

Unit - I Introduction and word level analysis

Dr. Saleha Saudagar

Team Members

AY 2026-27 SEM-II



Unit I - Syllabus



Unit I –Introduction and word level analysis 09 hours

- History, generic NLP system, levels and stages, ambiguity and challenges,
- words, corpora, morphology analysis, inflectional and derivational morphology,
- stemming, lemmatization, normalization, tokenization,
- bag of words, regular expression, finite automata, finite state transducers (FST), n grams language model.

History of NLP



The Natural Languages Processing started in the year 1940s.

1948 - In the Year 1948, the first recognisable NLP application was introduced in Birkbeck College, London.

1950s - In the Year 1950s, there was a conflicting view between linguistics and computer science. Now, Chomsky developed his first book syntactic structures and claimed that language is generative in nature.

In 1957, Chomsky also introduced the idea of Generative Grammar, which is rule based descriptions of syntactic structures.

(1960-1980) - Flavored with Artificial Intelligence (AI)

Introduction

- Humans communicate through some form of language either by text or speech.
- To make interactions between computers and humans, computers need to understand natural languages used by humans.
- Natural language processing is all about making computers learn, understand, analyze, manipulate and interpret natural(human) languages.
- NLP stands for Natural Language Processing, which is a part of Computer Science, Human language, and Artificial Intelligence.

Introduction

- The ability of machines to interpret human language is now at the core of many applications that we use every day - chatbots, Email classification and spam filters, search engines, grammar checkers, voice assistants, and social language translators.
- The input and output of an NLP system can be Speech or Written Text.

Introduction



- NLP is a subfield of AI that allows computers to understand, interpret, and translate human languages, as well as generate text. It aims to facilitate better communication between computers and humans by processing unstructured textual data.
- NLP is a broad field divided into two main areas: natural language understanding and natural language generation
- Natural Language Understanding (NLU) which involves transforming human language into a machine-readable format.
- It helps the machine to understand and analyse human language by extracting the text from large data such as keywords, emotions, relations, and e.

Introduction

- Natural Language Generation (NLG) acts as a translator that converts the computerized data into natural language representation.
- It mainly involves Text planning, Sentence planning, and Text realization.
- The NLU is harder than NLG

What is Natural Language Processing (NLP)



- The process of computer analysis of input provided in a human language (natural language), and conversion of this input into a useful form of representation.
- The field of NLP is primarily concerned with getting computers to perform useful and interesting tasks with human languages.
- The field of NLP is secondarily concerned with helping us come to a better understanding of human language.

NLP Application



Some NLP business applications include

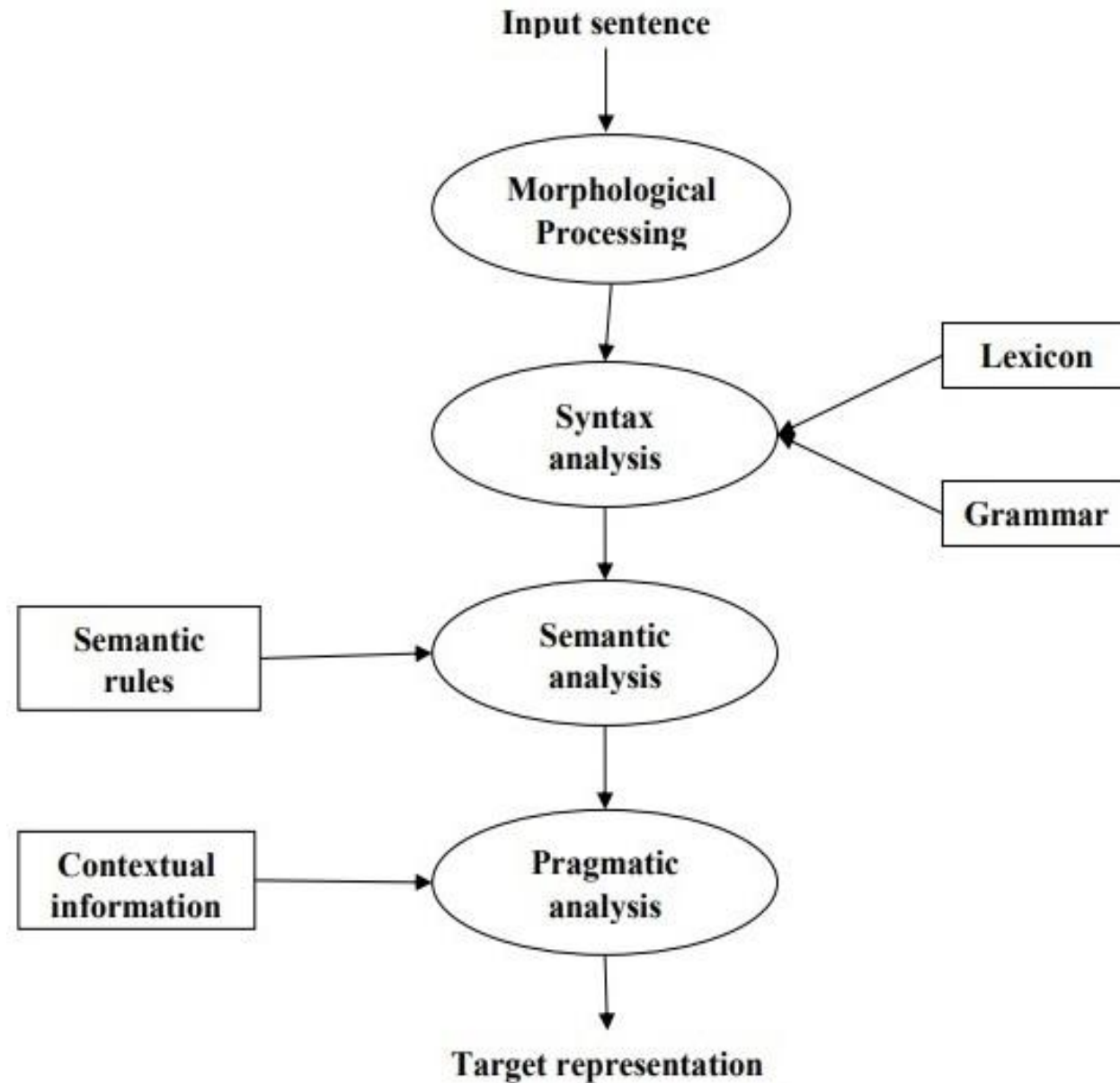
- **Text to speech**: Converting text-to-speech data, then reproducing the text as natural-sounding speech
- **Chatbots**: Helping chatbots understand and respond to customer inquiries
- **Urgency detection**: Analyzing language to prioritize tasks
- **Natural language understanding**: Converting speech to text and analyzing its intent
- **Autocorrect**: Detecting and removing text errors and suggesting alternatives
- **Sentiment analysis**: Revealing the perceptions people have of your goods and services and those of your competitors
- **Speech recognition**: Powering applications that understand users' voices and deciphering their meaning

Generic NLP system



- NLP is the branch of computer science focused on developing systems that allow computers to communicate with people using everyday language.
- Also called Computational Linguistics
- Also concerns how computational methods can aid the understanding of human language.

NLP Phases



The 5 Steps in Natural Language Processing (NLP)

1. Lexical analysis
2. Syntactic analysis
3. Semantic analysis
4. Discourse integration
5. Pragmatic analysis

NLP Terminology



Phonology – It is study of organizing sound systematically.

Morphology -The study of the formation and internal structure of words.

Morpheme – It is primitive unit of meaning in a language.

Syntax- The study of the formation and internal structure of sentences.

Semantics: The study of the meaning of sentences.

Pragmatics – It deals with using and understanding sentences in different situations and how the interpretation of the sentence is affected.

Discourse – It deals with how the immediately preceding sentence can affect the interpretation of the next sentence.

World Knowledge – It includes the general knowledge about the world.

1. Lexical analysis



Lexicon describes the understandable vocabulary that makes up a language.

Lexical analysis deciphers and segments language into units—or lexemes—like paragraphs, sentences, phrases, and words.

NLP algorithms categorize words into parts of speech (POS) and split lexemes into morphemes—meaningful language units that you can't further divide. There are 2 types of morphemes:

- **Free morphemes** function independently as words (like “cow” and “house”).
- **Bound morphemes** make up larger words. The word “unimaginable” contains the morphemes “un-” (a bound morpheme signifying a negative context), “imagine” (the free morpheme root of the whole word), and “-able” (a bound morpheme denoting the root morpheme’s ability to end).

1. Lexical analysis

For example, when performing a lexical analysis on the any paragraph, the analysis isolates and divides the first sentence into lexeme phrases. This analysis further divides the phrase into word lexemes, like “vocabulary” and “language,” categorizing both as noun POS. Then, the analysis derives free morphemes, like “words,” “vocabulary,” and “understand-,” and bound morphemes, like “-able.”

2. Syntactic analysis

- Syntax describes how a language's words and phrases arrange to form sentences.
- Syntactic analysis checks word arrangements for proper grammar.
- For instance, the sentence “Dave wrote the paper” passes a syntactic analysis check because it's grammatically correct.
- Conversely, a syntactic analysis categorizes a sentence like “Agra goes to the Poonam” as syntactically incorrect.

3. Semantic analysis

- Semantics describe the meaning of words, phrases, sentences, and paragraphs.
- Semantic analysis attempts to understand the literal meaning of individual language selections, not syntactic correctness.
- However, a semantic analysis doesn't check language data before and after a selection to clarify its meaning.
- For instance, "Manhattan calls out to Dave" passes a syntactic analysis because it's a grammatically correct sentence. However, it fails a semantic analysis. Because Manhattan is a place (and can't literally call out to people), the sentence's meaning doesn't make sense.

4. Discourse integration



- Discourse describes communication between 2 or more individuals.
- Discourse integration analyzes prior words and sentences to understand the meaning of ambiguous language.
- For example, if one sentence reads, “Manhattan speaks to all its people,” and the following sentence reads, “It calls out to Dave,” discourse integration checks the first sentence for context to understand that “It” in the latter sentence refers to Manhattan.

5. Pragmatic analysis



- Pragmatism describes the interpretation of language's intended meaning. Pragmatic analysis attempts to derive the intended—not literal—meaning of language.
- It helps you to discover the intended effect by applying a set of rules that characterize cooperative dialogues.
- **For Example:** "Open the door" is interpreted as a request instead of an order.

Ambiguity and Challenges



Ambiguity in NLP refers to sentences and phrases that potentially have two or more possible interpretations.

Types of Ambiguity

Lexical Ambiguity: This type of ambiguity represents words that can have multiple assertions. For instance, in English, the word “back” can be a noun (back stage), an adjective (back door) or an adverb (back away).

Syntactic Ambiguity: This type of ambiguity represents sentences that can be parsed in multiple syntactical forms. Take the following sentence: “ I heard his cell phone ring in my office”. The propositional phrase “in my office” can be parsed in a way that modifies the noun or on another way that modifies the verb.

Semantic Ambiguity: This type of ambiguity is typically related to the interpretation of sentence. For instance, the previous sentence used in the previous point can be interpreted as if I was physically present in the office or as if the cell phone was in the office.

words, corpora



What is Corpus in NLP?

- A corpus is a significant collection of texts written in everyday language that computers can read.
- When you have more than one, they're called 'corpora.'
- People make them from things like digital text, audio transcripts, and even scanned documents.
- Corpora are really important for studying and understanding how language is used in real life, just like people talk and write every day.

Why Do We Need Corpus in NLP?

A corpus is an essential tool for Natural Language Processing (NLP), serving as a fundamental resource.

A corpus is a significant, organized collection of text or audio data that often includes a wide range of documents, texts, or voices in one or more specific languages.

•Here are some points that show why the corpus is important in NLP:

- Training Machine Learning Models:** For a variety of NLP applications, including sentiment analysis, text classification, machine translation, and speech recognition, corpora are used to train and refine machine learning models. The massive amount of text data in the corpus is used to teach these models patterns, correlations, and complexities.
- Language Understanding:** Corpora gives a complete picture of the structure, grammar, vocabulary, and usage of a language. To learn how words and phrases are used in context and to generate new languages, NLP models utilize corpora.
- Rule-Based Systems:** Corpora are used by linguists and NLP experts to develop and test linguistic rules and patterns. Then, for tasks like part-of-speech tagging, grammatical processing, and named entity recognition, these rules are used in rule-based NLP systems.

- Lexicon and Semantics:** Lexicons, or dictionaries of words and their meanings, are created and expanded with the help of corpora. By showing word relationships, such as synonyms, antonyms, and word connections, they may help semantic analysis.
- Statistical Analysis:** Corpora are useful for language statistical analysis. They give information that is necessary for probabilistic NLP approaches to examine word frequency distributions, co-occurrence patterns, and other statistical features.
- Domain-Specific Knowledge:** Corpora are a source of domain-specific knowledge since they may be specific to particular topics or fields. Applications like the study of legal documents, the processing of medical records, and chatbots created for particular industries all depend on this.

Examples of Corpus in NLP



There are different types of corpora that are used in natural language processing (NLP), and each is designed for a particular linguistic research, machine learning, or computational language processing application. Some examples of corpora in NLP are given below:

•PennTreebank

- **Type:** Corpus with linguistic annotations
- **Description:** The Penn Treebank is an extensive collection of sentences that have been processed from sources like the Wall Street Journal. It is used for developing and evaluating analyzers and contains grammatical and part-of-speech annotations.

•WordNet

- **Type:** Corpus of lexical resources
- **Description:** WordNet is a lexical database that organizes words into synsets (sets of synonymous words) and shows the semantic connections between words. It is utilized for a variety of NLP applications, including information retrieval and word sense identification.

•VerbNet

- **Type:** Lexical Resource
- **Description:** A lexical database called VerbNet focuses on verbs and their themed functions. According to their grammatical and semantic behavior, it classifies verbs into groups known as verb classes. For a certain group of verbs, each verb class represents common usage patterns and argument structure.

•GloVe (Global Vectors for Word Representation)

- **Type:** Pre-trained word vectors
- **Description:** GloVe is a tool that offers pre-trained word embeddings, which identify the semantic connections between words. It is frequently used for sentiment analysis, word similarity, and raising the efficiency of NLP models.

Morphology analysis



- The study of word formation – how words are built up from smaller pieces.
- Morphology is the domain of linguistics that analyses the internal structure of words.
- Morphological analysis – exploring the structure of words
- Words are built up of minimal meaningful elements called morphemes:

For e.g.

played = play-ed ,

cats = cat-s ,

unfriendly = un-friend-ly

Two types of morphemes:

- i . Stems: play, cat, friend
- ii. Affixes: -ed, -s, un-, -ly

Two main types of affixes:

- i. Prefixes precede the stem: un-
- ii. Suffixes follow the stem: -ed, -s, un-, -ly

Morphology (both derivational and inflectional) encodes information that can be useful (or even essential) in NLP tasks

- Machine translation • Natural language understanding • Semantic role labeling

Inflectional Morphology



- Inflectional morphology conveys grammatical information, such as number, tense, agreement or case.
- Due to inflectional morphological process, the meaning and categories of the new inflected words usually do not change.
- That is, a noun can be inflected to a noun while adding affixes, a verb can be inflected to a verb in different tense in English.
- We would say, the root word (stem) is inflected to form other words of same meaning and category.
- Inflection creates different forms of the same word.

For example, the base word *bake* in *baking* still communicates the idea of cooking with flour and eggs using an oven. The suffix *-ing* simply adjusts the tense to express when it happens.

Category	Stem	Affixes	Inflected word
Noun	word	-s	words
	box	-es	boxes
Verb	treat	-s	treats
		-ing	treating
		-ed	treated

Derivational Morphology

- Derivational morphemes can create new words when added to a base word, but inflectional morphemes cannot.
- Derivational morphemes derive new words by adding an affix—much like an inflectional morpheme—but they are capable of changing the base word's grammatical category, which inflectional morphemes cannot do.
- E.g
 - Caramel (noun) + ize = caramelize (verb)
 - Decorate (verb) + ive = decorative (adjective)
 - Flour (noun) + less = flourless (adjective)

- When a derivational morpheme is added to a base word, that word might (not in every case) take a new meaning entirely.
- While these words are certainly related in essence, the word class has changed and thus altered the way the word functions.
- When an inflectional morpheme is added to a base word, the base simply becomes inflected. This means that the word changes forms to reflect its grammatical surroundings.

Introduction to Text Analysis: Text-preprocessing,

- In any Machine learning task, cleaning or preprocessing the data is as important as model building.
- Text data is one of the most unstructured forms of available data and when comes to deal with Human language then it's too complex.
- Alexa, Siri, Google assistant can understand, process, and respond in Human language.
- NLP is a technology that works behind it where before any response lots of text preprocessing takes place.

- Natural Language Processing is a branch of Artificial Intelligence that analyzes, processes, and efficiently retrieves information text data.
- NLP one can solve a huge range of real-world problems which include summarizing documents, title generator, caption generator, fraud detection, speech recognition, recommendation system, machine translation, etc
- **Text Preprocessing**
- Text preprocessing is a method to clean the text data and make it ready to feed data to the model.
- Text data contains noise in various forms like emotions, punctuation, text in a different case.

Libraries used to deal with NLP problems

There are many libraries and algorithms used to deal with NLP-based problems.

A regular expression(re) is mostly used library for text cleaning.

NLTK(Natural language toolkit) and spacy

Techniques for Text Preprocessing

1. Understanding Problem Statement
2. Load data

1) Expand Contractions

- Contraction is the shortened form of a word like **don't** stands for **do not**, **aren't** stands for **are not**.
- We need to expand this contraction in the text data for better analysis.
- You can easily get the dictionary of contractions on google or create your own and use the re module to map the contractions.

2) Lower Case

- If the text is in the same case, it is easy for a machine to interpret the words because the lower case and upper case are treated differently by the machine.
- for example, words like Ball and ball are treated differently by machine.
- So, we need to make the text in the same case and the most preferred case is a lower case to avoid such problems

3) Remove punctuations

- One of the other text processing techniques is removing punctuations. there are total 32 main punctuations that need to be taken care of.
- we can directly use the string module with a regular expression to replace any punctuation in text with an empty string.
- 32 punctuations which string module provide us is listed below.

'!"#\$%&'()*+,-./:;<=>?@[\\]^_`{|}~'

4) Remove words and digits containing digits



- Sometimes it happens that words and digits combine are written in the text which creates a problem for machines to understand.
- We need to remove the words and digits which are combined like **game57** or **game5ts7**.
- This type of word is difficult to process so better to remove them or replace them with an empty string.
- we use regular expressions for this.

5) Remove Stopwords

- Stopwords are the most commonly occurring words in a text which do not provide any valuable information.
- stopwords like they, there, this, where, etc are some of the stopwords.
- NLTK library is a common library that is used to remove stopwords and include approximately 180 stopwords which it removes.
- If we want to add any new word to a set of words then it is easy using the add method.

7) Stemming and Lemmatization



- Stemming is a process to reduce the word to its root stem for example run, running, runs, runed derived from the same word as run.
- Basically stemming do is remove the prefix or suffix from word like ing, s, es, etc.
- NLTK library is used to stem the words.
- The stemming technique is not used for production purposes because it is not so efficient technique and most of the time it stems the unwanted words.
- So, to solve the problem another technique came into the market as Lemmatization. there are various types of stemming algorithms like porter stemmer, snowball stemmer.
- Porter stemmer is widely used present in the NLTK library.

Lemmatization is similar to stemming, used to stem the words into root word but differs in working.

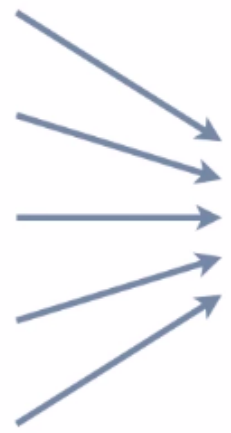
Lemmatization is a systematic way to reduce the words into their lemma by matching them with a language dictionary.

Form	Suffix	Stem
studie s	-es	studi
study ing	-ing	study
niñ as	-as	niñ
niñ ez	-ez	niñ

Stemming

Stemming vs Lemmatization

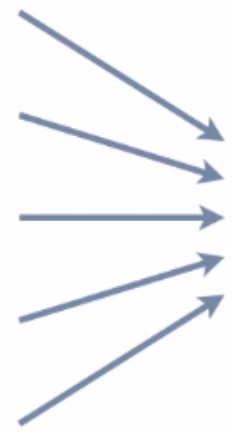
change
changing
changes
changed
changer



chang

The diagram illustrates the stemming process. Five arrows point from the words 'change', 'changing', 'changes', 'changed', and 'changer' on the left to the word 'chang' on the right. The word 'chang' is colored blue.

change
changing
changes
changed
changer



change

The diagram illustrates the lemmatization process. Five arrows point from the words 'change', 'changing', 'changes', 'changed', and 'changer' on the left to the word 'change' on the right. The word 'change' is colored green.

Tokenization

- Tokenization is the process of breaking down the given text in natural language processing into the smallest unit in a sentence called a token.
- Punctuation marks, words, and numbers can be considered tokens.

why do we need Tokenization?

[Tokenization](#) is a foundation step in NLP pipeline that shapes the entire workflow.

Involves dividing a string or text into a list of smaller units known as tokens.

Uses a tokenizer to segment unstructured data and natural language text into distinct chunks of information, treating them as different elements.

Tokens: Words or Sub-words in the context of natural language processing.

"This is a sample"

Tokenization

"This" "is" "a" "sample"

Types of Tokenization in NLP



Popular Approaches of Tokenization



Bag of words

- Natural Language Processing technique of text modeling known as **Bag of Words** model.
- Whenever we apply any algorithm in NLP, it works on numbers.
- We cannot directly feed our text into that algorithm.
- Hence, Bag of Words model is used to preprocess the text by converting it into *a bag of words*.
- *BoW is to* keeps a count of the total occurrences of most frequently used words.

Example of BoW

Sentence 1: "Welcome to Great Learning, Now start learning"

Sentence 2: "Learning is a good practice"

Step 1: Go through all the words in the above text and make a list of all of the words in our model vocabulary.

Welcome , To ,Great , Learning , , , Now , start , learning ,is ,a ,good ,practice

vocabulary has 12 words, we can use a fixed-length document-representation of 12, with one position in the vector to score each word.

Word	Frequency
Welcome	1
to	1
Great	1
,	1
Now	1
start	1
learning	3
is	0
a	0
good	0
practice	0

Writing the above frequencies in the vector

Sentence 1 $\rightarrow$ [
 1,1,1,1,1,1,1,1,0,0,0]

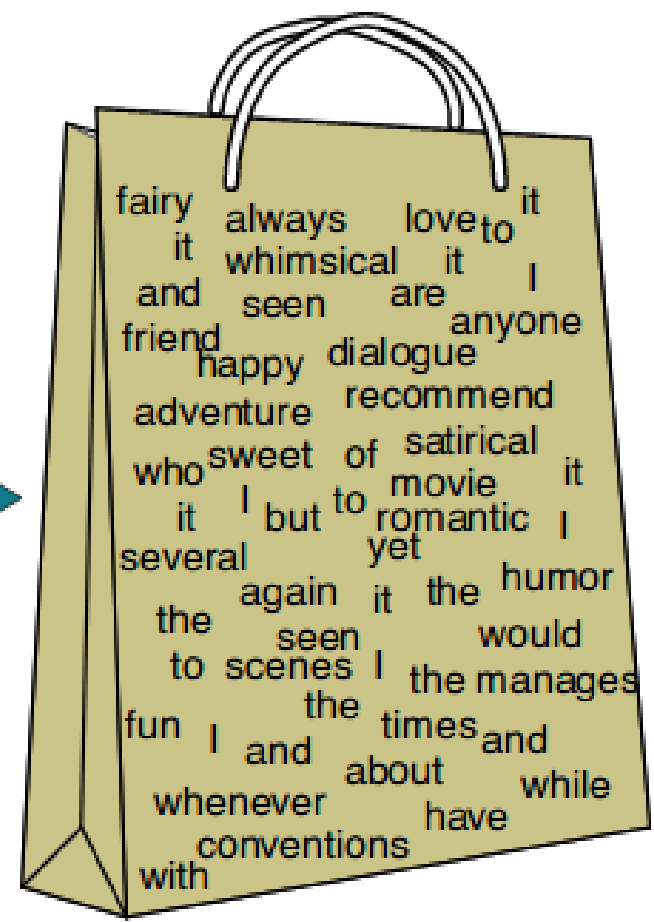
Now for sentence 2, the scoring would like

Word	Frequency
Welcome	0
to	0
Great	0
Learning	1
,	0
Now	0
start	0
learning	0
is	1
a	1
good	1
practice	1

Similarly, writing the above frequencies in the vector form

Sentence 2 $\rightarrow$ [0,0,0,0,0,0,0,1,1,1,1,1]

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
...	...

	about	bird	heard	is	the	word	you
About the bird , the bird , bird bird bird	1	5	0	0	2	0	0
You heard about the bird	1	1	1	0	1	0	1
The bird is the word	0	1	0	1	2	1	0

Example of BoW with preprocessing

Sentence 1: "Welcome to Great Learning, Now start learning"

Sentence 2: "Learning is a good practice"

Step 1: Convert the above sentences in lower case.

Step 2: Remove special characters and stopwords from the text.

After applying the above steps, the sentences are changed to

Sentence 1: "welcome great learning now start learning"

Sentence 2: "learning good practice"

Step 3: make a list of all of the words in our model vocabulary.

Welcome ,great,learning,now,start,good,practice

frequencies in the vector form

Sentence 1: "welcome great learning now start learning"

Sentence 1 $\rightarrow$ [1,1,2,1,1,0,0]

Sentence 2: "learning good practice"

Sentence 2 $\rightarrow$ [0,0,1,0,0,1,1]

Example 2

- Review 1: This movie is very scary and long
- Review 2: This movie is not scary and is slow
- Review 3: This movie is spooky and good

Ans

vocabulary consists of these 11 words: 'This', 'movie', 'is', 'very', 'scary', 'and', 'long', 'not', 'slow', 'spooky', 'good'.

Vector-of-Review1: [1 1 1 1 1 1 1 0 0 0 0]

Vector-of-Review2: [1 1 2 0 0 1 1 0 1 0 0]

Vector-of-Review3: [1 1 1 0 0 0 1 0 0 1 1]

Drawbacks of using a Bag-of-Words (BoW) Model



In the above example, we can have vectors of length 11. However, we start facing issues when we come across new sentences:

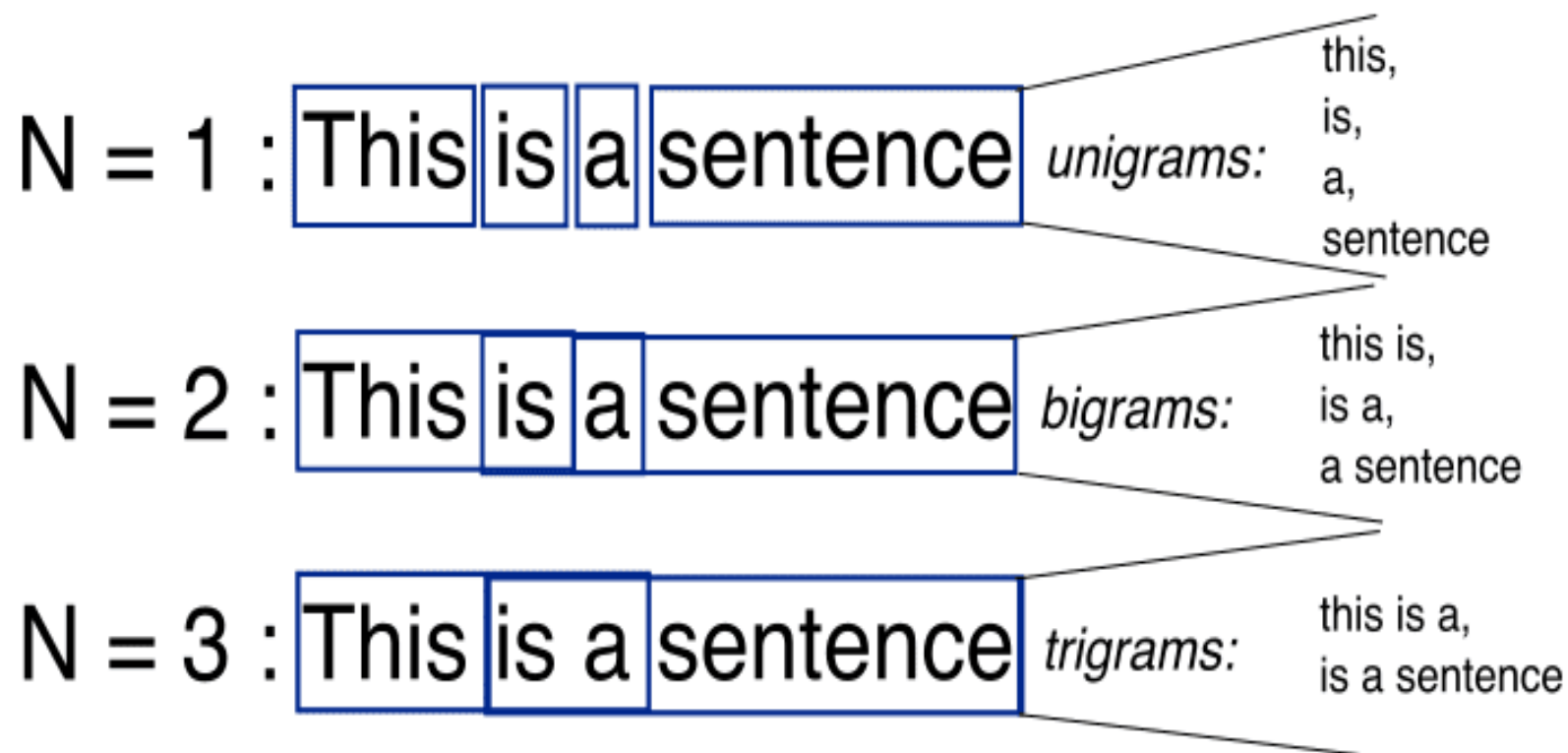
1. If the new sentences contain new words, then our vocabulary size would increase and thereby, the length of the vectors would increase too.
2. Additionally, the vectors would also contain many 0s.
3. We are retaining no information on the grammar of the sentences nor on the ordering of the words in the text.

n grams language model

- Language modeling is used to determine the probability of the word's sequence.
- It is a fundamental task in natural language processing.
- This modeling has a large number of applications i.e. recognition of speech, filtering of spam, etc
- An N-gram is a contiguous sequence of N items (words, characters, or other tokens) from a given sample of text or speech.

N-gram Modelling in NLP

- N-gram is a sequence of the N-words in the modeling of NLP.
- Consider an example of the statement for modeling. “I love reading history books and watching documentaries”.
- In one-gram or unigram, there is a one-word sequence. As for the above statement, in one gram it can be “I”, “love”, “history”, “books”, “and”, “watching”, “documentaries”.
- In two-gram or the bi-gram, there is the two-word sequence i.e. “I love”, “love reading”, or “history books”.
- In the three-gram or the tri-gram, there are the three words sequences i.e. “I love reading”, “history books,” or “and watching documentaries”.



How do N-gram Language Models work?

An N-gram language model predicts the probability of a given N-gram within any sequence of words in the language. If we have a good N-gram model, we can predict $p(w \mid h)$ – what is the probability of seeing the word w given a history of previous words h – where the history contains $n-1$ words.

Advantages of N-grams



1.Simplicity: N-grams are intuitive and relatively simple to understand and implement.

2.Low Memory Usage: They require minimal memory for storage compared to more complex models.

Limitations of N-grams

1.Limited Context: N-grams have a finite context window, which means they cannot capture long-range dependencies or context beyond the previous $N-1$ words.

2.Sparsity: As N increases, the number of possible N-grams grows exponentially, leading to sparse data and increased computational demands.

N-grams in NLP are used for:

- **Capturing Context and Semantics:** N-grams help us understand how words work together in a sentence. By analyzing small word combinations they provide insight into the meaning and flow of language making text interpretation more accurate.
- **Improving Language Models:** In tools like translation systems or voice assistants N-grams help create smarter models that can better guess what comes next in a sentence, leading to more natural and accurate responses.
- **Enhancing Text Prediction:** They are widely used in predictive typing. By analyzing the words you've already typed they help suggest what you're likely to type next making writing faster and more intuitive.
- **Information Retrieval:** When searching for information they help to find and rank documents by recognizing important word patterns. This makes search engines more effective at delivering relevant results.

```
def generate_ngrams(text, n):  
    tokens = text.split()  
    ngrams = [tuple(tokens[i:i + n]) for i in range(len(tokens) - n + 1)]  
    return ngrams
```

```
text = "NAtural language processing"
```

```
unigrams = generate_ngrams(text, 1)
```

```
bigrams = generate_ngrams(text, 2)
```

```
trigrams = generate_ngrams(text, 3)
```

```
print("Unigrams:", unigrams)
```

```
print("Bigrams:", bigrams)
```

```
print("Trigrams:", trigrams)
```

Regular expression

- Regular expression (often shortened to regex), a language for specifying text search regular expression strings.
- Formally, a regular expression is an algebraic notation for characterizing a set of strings.
- Regular expressions are particularly useful for searching in texts, when we have a pattern to search for and a corpus of texts to search through.
- A regular expression search function will search through the corpus, returning all texts that match the pattern.

- Regular Expressions are used in various tasks such as data pre-processing, rule-based information mining systems, pattern matching, text feature engineering, web scraping, data extraction, etc.

Regular expression sets can be used to match either a pattern or another pattern.

[abc] : Matches either a, b, or c. It does not match abc;

[a-z] : Matches any alphabet from a to z;

[A-Z] : Matches any alphabets in capital from A to Z;

[a\-p] : Matches a, -, or p. It matches - because \ escapes it;

[-z] : Matches - or z;

[a-z0–9] : Matches characters from a to z or from 0 to 9.

Examples of Regular Expressions

Regular Expressions

Regular Set

$(0 + 10^*)$

$\{0, 1, 10, 100, 1000, 10000, \dots\}$

(0^*10^*)

$\{1, 01, 10, 010, 0010, \dots\}$

$(0 + \epsilon)(1 + \epsilon)$

$\{\epsilon, 0, 1, 01\}$

$(a+b)^*$

It would be set of strings of a's and b's of any length which also includes the null string i.e. $\{\epsilon, a, b, aa, ab, bb, ba, aaa, \dots\}$

$(a+b)^*abb$

It would be set of strings of a's and b's ending with the string abb i.e. $\{abb, aabb, babb, aaabb, ababb, \dots\}$

$(11)^*$

It would be set consisting of even number of 1's which also includes an empty string i.e. $\{\epsilon, 11, 1111, 111111, \dots\}$

$(aa)^*(bb)^*b$

It would be set of strings consisting of even number of a's followed by odd number of b's i.e. $\{b, aab, aabbb, aabbbb, aaaab, aaaabb, \dots\}$

$(aa + ab + ba + bb)^*$

It would be string of a's and b's of even length that can be obtained by concatenating any combination of the strings aa, ab, ba and bb including null i.e. $\{aa, ab, ba, bb, aaab, aaba, \dots\}$

finite automata



The regular expression is more than just a convenient metalanguage for text searching.

First, a regular expression is one way of describing a finite-state automaton (FSA).

Finite state automata (deterministic and nondeterministic finite automata) provide decisions regarding the acceptance and rejection of a string while transducers provide some output for a given input.

Any regular expression can be implemented as a finite-state automaton.

Symmetrically, any finite-state automaton can be described with a regular expression

Second, a regular expression is one way of characterizing a particular kind REGULAR LANGUAGE of formal language called a regular language.

Both regular expressions and finite state automata can be used to describe regular languages.

Finite Automata in NLP

1. Finite State Transducer

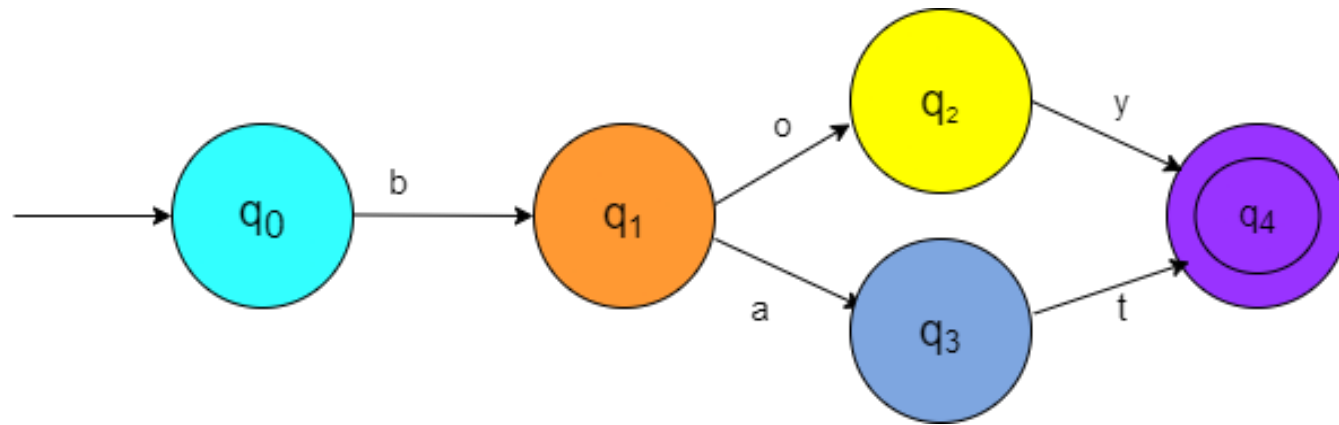
A finite-state transducer (FST) is a finite-state machine with two memory tapes. An FST is a type of finite-state automaton (FSA) that maps between two sets of symbols.

2. Language Recognizer

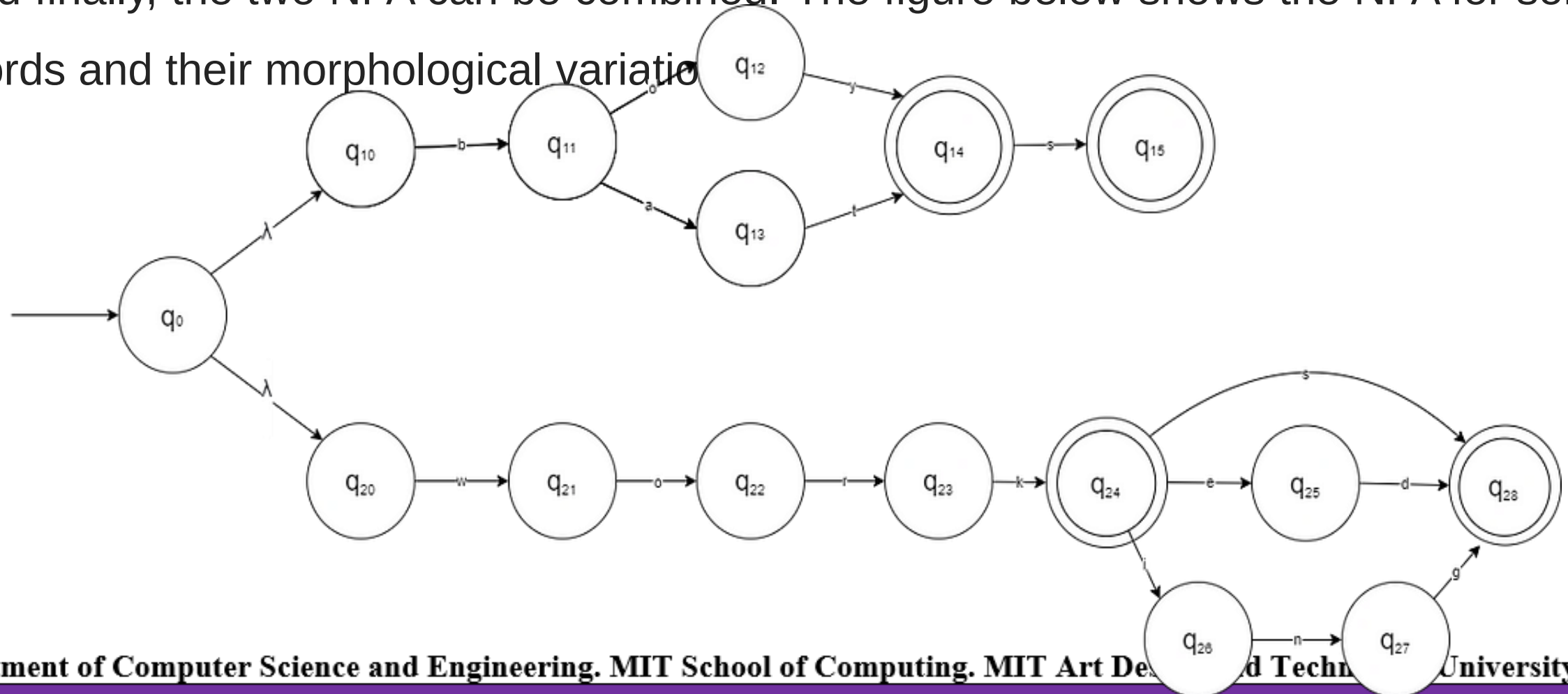
There are many tasks that need language recognizing mechanism.

For example, spelling checker, morphological analysis, language identification etc.. Finite state machine are quite useful as a language recognizer.

For example, NFA for the words 'boy' and 'bat' is shown in the Figure below. Similarly for every word a NFA can be designed and the different NFA's can be combined to form spelling checker or dictionary compilation for a language.



For each category of words, we can form a separate NFA and then combine them using transitions. For example, nouns and their plural can be recognized through one NFA and verbs and their different forms can be recognized through another NFA and finally, the two NFA can be combined. The figure below shows the NFA for some words and their morphological variations.



Challenges in NLP



1. Contextual words and phrases and homonyms

Homonyms – two or more words that are pronounced the same but have different definitions – can be problematic for question answering and speech-to-text applications because they aren't written in text form. Usage of *their* and *there*.

2. Ambiguity

Ambiguity in NLP refers to sentences and phrases that potentially have two or more possible interpretations.

3. Errors in text and speech

Misspelled or misused words can create problems for text analysis. Autocorrect and grammar correction applications can handle common mistakes, but don't always understand the writer's intention.

4. Development Time and Resource Requirements

Development Time and Resource Requirements for ***Natural Language Processing (NLP)*** projects depends on various factors consisting the task complexity, size and quality of the data, availability of existing tools and libraries, and the team of expert involved.

5. Real-Time Processing and Responsiveness

For applications like digital assistants or real-time translation services, the ability to process language in real time is crucial. Minimizing latency while maintaining accuracy is a challenging aspect of NLP.

6. Data Quality and Availability

The effectiveness of NLP models is heavily dependent on the quality and quantity of the training data.

7. Integrating NLP with Existing IT Infrastructure

Integrating Natural Language Processing into existing IT infrastructure is a strategic process that requires careful planning and execution. This integration can significantly enhance the capability of businesses to process and understand large volumes of language data, leading to improved decision-making, customer experiences, and operational efficiencies.